

Egyszerű adattípusok: egész, valós, logikai és karakter típusok és kifejezések. Az egyszerű típusok reprezentációja, számábrázolási tartományuk, pontosságuk, memória igényük és műveleteik. Az összetett adattípusok és a típusképzések, valamint megvalósításuk C nyelven. A pointer, a tömb, a rekord és az unió típus. Az egyes típusok szerepe, használata.

MŰVELETEK ÉS KIFEJEZÉSEK

- **Műveletek típusai**

- a műveleti jeleket (operátorokat) a C nyelvben az operandusok száma és a funkciójuk szerint csoportosítjuk
- **Operandusok száma szerint**
 - egyoperandusú (unary): negálás(-v), logikai nem (!), címképzés (&), indirekció (*p), sizeof() és értékadás (=)
 - kétoperandusú (binary): aritmetikai műveletek (+, -, %, /, *) és relációs műveletek (>, <, stb)
 - háromoperandusú (ternary): feltételes művelet: feltétel ? igaz_ág : hamis_ág
- **Funkció szerint**
 - **aritmetikai**: +, -, *, / (egészeknél bennfoglalás, valósoknál osztás), % (maradékképzés - csak egészeknél!)
 - **relációs** (összehasonlító): ==, !=, <, >, <=, >=
 - eredményük int: 1 ha igaz, vagy 0 ha hamis
 - **logikai**: && (és), || (vagy), ! (nem)
 - **bitműveletek**: & (bitenkénti és), | (bitenkénti vagy), ^ (kizáró vagy), ~ (komplement), <<, >> (léptetések)
 - **léptető műveletek**: ++, --, lehet prefix(++i) és postfix(i++)
 - **értékadó**: =, +=, -=, *=, /=, %=, stb

- **Kifejezések kiértékelése**

- ha nincsenek zárójelek akkor 2 fő szabály határozza meg
- **Prioritás (precedencia)**
 - műveletek erősségét határozza meg
 - pl: * erősebb a +-nál, így az $a + b * c$ jelentése $a + (b * c)$
 - legerősebbek: zárójelek, tömbindexelés [], struktúra elérés ., ->
 - leggyengébbek: vessző operátor
- **Asszociativitás**
 - ha azonos precedenciájú műveletek állnak egymás mellett ez dönti el melyik jön előbb
 - **balról jobbra**
 - legtöbb művelet ilyen (+, -, *, /)
 - pl: $10 - 5 - 2$ jelentése $(10 - 5) - 2 = 3$
 - **jobbról balra**
 - egyoperandusú műveletek pl: !, ++, sizeof, értékadás (=), stb...
 - pl: $a = b = c = 5$ jelentése $a = (b = (c = 5))$

ADATTÍPUSOK

- az adattípus olyan egysége a programnak, melyet két dolog határoz meg, az **értékhalmaza** és ezen értékeken végezhető **műveletekhalmaza**
- 2 nagy csoportja van a típusoknak
 - **elemi (egyszerű) típusok**
 - értékeiket már nem lehet önmagukban értelmes részekre bontani
 - létezésük a létrehozás blokkjához kötött
 - ide tartoznak a **szám típusok (egész, valós)**, **karakterek**, **logikai értékek** és a **felsorolás típus (enum)** is
 - **összetett (kompozit) típusok**
 - más egyszerűbb adattípusokból épülnek fel valamilyen konstrukciós elv (pl: sorozat, direkt szorzat) alapján
 - ide tartoznak a **tömbök**, **rekordok (struct)**, **uniók** és a **pointerek** is

EGYSZERŰ ADATTÍPUSOK (ELEMI ADATTÍPUSOK)

- **Egész típus - int**
 - **Reprezentáció**
 - a számítógép a memóriában bitek sorozataként tárol mindent (bináris számok)
 - egész számok esetén megkülönböztetünk **előjeles (signed)** és **előjeltelen (unsigned)** ábrázolást
 - **előjeltelen (unsigned)**
 - k bites memória esetén az értékkészlete: $[0, 2^k - 1]$
 - **előjeles (signed)**
 - a legelső bit lesz az előjelbit (0: ha pozitív, 1: ha negatív)
 - k bites memória esetén az értékkészlete: $[-2^{k-1}, 2^{k-1} - 1]$
 - **kettes komplement** módon történik a reprezentáció
 - pozitív szám kettes komplemente mindig önmaga
 - negatív szám esetén:
 - előjelet elhagyjuk és binárisra alakítjuk
 - a szám minden bitjét negáljuk ($1 \rightarrow 0, 0 \rightarrow 1$)
 - ez a szám egyes komplemente
 - majd hozzáadunk egyet és megkapjuk a szám kettes komplementjét
 - előnye, hogy a gépnek nem kell különbséget tennie az összeadás és kivonás logikája között, valamint a 0-nak egyetlen ábrázolása van
 - a szabvány relatív méreteket köt ki, de a konkrét méret implementáció függő
 - $\text{sizeof}(\text{short}) \leq \text{sizeof}(\text{int}) \leq \text{sizeof}(\text{long}) \leq \text{sizeof}(\text{long long})$,
lényeg, hogy ennek megfelelően, pl short mindig kisebb mint int

- **Típusai**
 - **char, int**
 - az int értelmezési tartományát különböző módosítókkal (signed, unsigned, short, long, long long) módosíthatjuk
 - long long int = long long vagy short int = short, mindegy melyik formát használjuk
- **Memóriaigény**
 - 8 bit: **char, unsigned char, signed char**
 - 16 bit: **short int, signed short int, unsigned short int**
 - 32 bit
 - **int, signed int, unsigned int**
 - **long int, signed long int, unsigned long int**
 - 64 bit: **long long int, signed long long int, unsigned long long int**
- **Pontosság**
 - alulcsordulás és túlcsordulás is előfordulhat, ha a reprezentált érték az értéktartományon kívülre esik
 - de ez nem mindig okoz hibát a programban, de az eredmény helytelen lehet
- **Műveletek**
 - minden aritmetikai és relációs művelet működik rájuk
- **Karaktertípus - char**
 - **Reprezentáció**
 - a karaktereket betűként kezeljük, de a **gép számára ezek is kis egész számok**, a hozzárendelést a karakterkódolás (pl: ASCII) adja meg
 - '5' és 5 között az a különbség, hogy '5' egy karakter, aminek ASCII kódja 53, míg a sima 5 az egy konkrét numerikus érték
 - literálok
 - közvetlen megadás: 'a', '5'
 - escape-szekvenciák: '\n' (újsor), '\t' (tabulátor), '\0' (null karakter, a sztringek vége)
 - **Memóriaigénye: 1 bájt (8 bit)**
 - **Számábrázolási tartomány**
 - értelmezési tartománya módosítható módosítókkal (signed, unsigned)
 - **char:** fordítófüggő
 - **signed char:** [-128, 127]
 - **unsigned char:** [0, 255]
 - **Pontosság**
 - mivel számkként kezel a gép ezért az alulcsordulás és a túlcsordulás is előfordulhat, ha a reprezentált érték az értéktartományon kívülre esik
 - **Műveletek**
 - szintúgy minden aritmetikai és relációs művelet működik rájuk mint a többi egész típusra

- **Valós típus - float, double, long double**

- **Reprezentáció**

- ábrázolására a lebegőpontos formátumot használjuk

- Előjelbit (0 = pozitív, 1 = negatív)

- Exponens (kitevő)

- meghatározza a nagyságrendet (értéktartomány méretét)

- karakterisztika

- Mantissza (tört, a szám értékes jegyei)

- meghatározza a szám jegyeit, azaz a pontosságot

- a valós értékeket mindenképp előjelesen kezeljük

- **Típusai, azok Memória igénye és Pontossága**

- **float** (egyszeres pontosság)

- általában 32 bit memóriaigény

- 1 bit előjel, 8 bit exponens és 23 bit mantissza

- pontossága kb 6-7 tizedesjegy

- **double** (kétszeres pontosság)

- általában 64 bit memóriaigény

- 1 bit előjel, 11 bit exponens és 52 bit mantissza

- pontossága 15-16 tizedesjegy

- **long double**

- ez egy kiterjesztett pontosság, a rendszertől függ

- 64 bit, mint a sima double (windows)

- 80 bit = 1 bit előjel, 15 bit exponens és 55 bit mantissza

- 128 bit = 1 bit előjel, 15 bit exponens és 112 bit mantissza

- **Műveletek**

- azonosak az egész típusok műveleteihez csupán figyelni kell arra, hogy a logikai ==, != műveletek esetén ha **float**-ot hasonlítunk össze **double**-el akkor a pontosság miatt lehet eltérés a kettő érték között

- **Felsorolás típus - enum**

- **Reprezentáció**

- olyan speciális **egész értékeket** megadó típus a C nyelvben, amelyben a típus elemeit egy felsorolás határozza meg

- **typedef enum {hetfo, kedd, szerda, csutortok, pentek, szombat, vasarnap} het_tipus;**

- a felsorolásban az egyes elemeket egy-egy azonosítóval adjuk meg egymás után, a felsorolás első elemének értéke 0, a többi pedig egyel több mint az öt megelőző elemnek

- akár konkrét értéket (**csak egész**) is meg tudunk adni:

- enum ertekek_t { a = 3, b, c, d = 2, e, f };

- **Memória igénye:** 32 bit (tehát mint egy int), bizonyos implementációkba ha kicsik a számok akkor short-ra vagy char-ra is konvertálhatja, hogy kisebb helyet foglaljon

- **Logikai típus - bool**
 - **Reprezentáció**
 - eredetileg a C-ben nem definiált adattípus, de a C99-es szabvánnyal bekerült, használatához szükséges a *stdbool.h* header file-t behúzni
 - de megoldható a megvalósítása úgy is, hogy a 0 a false, ha más van ott akkor true
 - **Memóriaigénye:** 8 bit
 - igazából 1 bites, de egy változót 1 byte (8bit) alatt nem tudunk tárolni, így muszáj 8 biten
 - **Értékkészlete:** [true, false]
 - **Pontosság**
 - elég pontos, ilyen bináris értékeket nem könnyű elrontani
 - **Műveletek**
 - negálás (!), egyenlő (==), nem egyenlő (!=), logikai éselés (&&), logikai vagyolás (||)
 - a logikai kifejezések kiértékelése "**rövidzár**" (**short-circuit**) módon történik
 - az A && B esetén A hamis, B-t meg sem nézi a gép

ÖSSZETETT ADATTÍPUSOK

- **Pointer**
 - olyan változó, mely nem közvetlenül tárolja az értéket, hanem egy másik memória területre mutat (például egy másik adat, változó memória címét tárolja)
 - **dinamikus változó**
 - ha egy változó nem függ az őt tartalmazó blokk aktivitásától akkor dinamikus változóról van szó
 - egy pointer típusú változó értéke egy meghatározott dinamikus változó
 - **Memóriaigény**
 - architektúrától függ, tehát 32 bites rendszeren 4 bájtot (32bit), míg 64 bites rendszeren 8 bájtot (64bit) foglal el függetlenül attól, hogy milyen típusra mutat
 - **Deklarációja:** típus *valtozo_nev; (pl: int *p;)
 - fontos, hogy int *p; esetén:
 - *p egy int típusú dinamikus változó
 - p egy int* típusú, azaz egy int-re mutató változó
 - **Műveletek**
 - **Címképzés (&)**
 - megadja egy létező változó címét:

```
int i = 5;
int *p = &i;
```
 - magas precedencia

■ Indirekció / Dereferencia (*)

- eléri az értéket ami a pointer által mutatott memóriacímen található
- magas precedencia

■ Cím foglalás (létesítés)

- dinamikus címfoglalás, új cím foglалódik
- `int *p = malloc(sizeof(típus))`

■ NULL

- ha nem adunk neki értéket, akkor nincs definiálva hova mutat, de hibát nem ad (**null pointer** egy speciális konstans)

■ Értékadás

- másik változó címét adjuk neki (címképzéssel)
 - `int i = 5;`
`int *p = &i;`
 - ekkor az adott memória területnek 2 címe lesz, ha az egyik értéke változik akkor a másik is mivel ugyanarra a területre mutat

■ Törlés (free())

- az általa hivatkozott dinamikus változó törlésre kerül és innentől semmire nem hivatkozik
 - fontos, hogy csak dinamikusan foglalt változóra jó, tehát stack-en lévő lokális változóra nem és felesleges is
- `int *p = malloc(sizeof(int));`
`free(p);`
`p = NULL;`

■ Egyenlő (==)

- összehasonlítja, hogy a két pointer ugyanarra a dinamikus változóra, memória címre mutat-e (vagy, hogy mindkettő NULL-e)

■ Nemegyenlő (!=)

- összehasonlítja, hogy a két pointer másik dinamikus változóra, memória címre mutat-e (beleértve, hogy az egyik legfeljebb NULL)

● Tömb (Array)

- azonos típusú elemek véges sorozata, amelyek a memóriában folytonosan (egymás után) helyezkednek el egy összefüggő memória területen
- **Deklarációja:** `típus változo_nev[elemszám];` (pl: `int t[10];`)
- **Indexelés:** mindig a 0-tól indul, ha `t[10]`, azaz 10 elemű akkor: `t[0], ..., t[9]`
 - C-ben **nincs automatikus indexhatár-ellenőrzés**, azaz ha túlindexeljük a tömböt akkor a program más memóriaterületet, olvashat/írhat felül
 - biztonsági hiba

- **Többdimenziós tömbök**
 - tömbök tömbje, pl: `int m[3][3]`; egy mátrix
 - tárolása sorfolytonos (lexikográfiai sorrend)
 - `m[0]` sorhoz tartozó összes elem (`m[0][0]`, ... `m[0][j]`) egymás után, majd `m[1]`-es sorhoz tartozók és így tovább
- Karakterek tömbjét stringnek hívjuk, amelyet C-ben egy speciális escape-szekvencia a `'\0'` (null) zár le
- **Műveletek**
 - **Kiolvas:** `t[i]`; és **Módosít:** `t[i] = 5`; műveletek
 - a kiolvasás és módosítás is tömbelem-hivatkozással történik, melyre a `[]` operátor való
 - magas precedencia és balról jobbra asszociatív
 - **Másolás**
 - a C-ben a tömbnév önmagában egy konstans pointer az első elemre, ezért az `A = B` értékadás nem másolja az elemeket
 - ehelyett a `memcpy()` függvényt vagy ciklust kell használni
- **Pointer és Tömb kapcsolata**
 - a **tömb neve valójában egy konstans pointer**, mely a tömb első elemére mutat (0. index)
 - tehát a tömb neve egy nem módosítható (konstans) pointer
 - **pointer-aritmetika** (hozzáadás/kivonás, a típus méretével lép)
 - ha egy `p` pointer egy tömb elemére mutat, akkor a `p + 1` a következő elem címére mutat (a gép automatikusan a típus méretével léptet, melyet a `sizeof`-val kap meg)
 - ebből kijön az **aritmetikai összefüggés**, hogy `t[i]` matematikailag ugyanaz, mint a `*(t+i)`
 - ha egy tömböt adunk át egy függvény paraméterének akkor igazából csak az első elem címét adjuk át, azaz a tömb "leépül" pointerre
- **Rekord (struktúra - struct)**
 - akkor használjuk ha különböző típusú (heterogén), de logikailag összetartozó adatokkal kell dolgozni
 - matematikailag ez a típusok direkt szorzata
 - **Definíció és deklaráció**

```

struct Hallgato {
    char nev[50];
    int neptun_kod;
    double atlag;
};
          
```

 - struktúrán belüli változók a **mezők (tagok)**
 - deklaráció: **struct Hallgato h1;**
 - C-ben a struct kulcsszó kiírása a típusnév előtt kötelező, hacsak nem használunk `typedef`-et

- **Műveletek** (a fenti példára)
 - **Elérés**
 - a pont operátorral, így tudjuk olvasni és módosítani is egy adott komponens értékét
 - pl: `h1.nev`; vagy `h1.atlag = 4.5`
 - **Értékadás**
 - két azonos típusú struktúra közvetlenül értékül adható egymásnak (`h2 = h1;`)
 - ilyenkor "sekély másolás" történik (bitről bitre másolódik a tartalom)
- **Memóriarendezés**
 - a memóriában a mezők a deklaráció sorrendjében egymás után helyezkednek el
 - **igazítás (alignment)**
 - a processzor hatékonyabban éri el az adatokat, ha azok bizonyos memóricímeken kezdődnek (pl: egy 4 bájtos int címe legyen 4-el osztható)
 - **kitöltő bájtok (padding)**
 - az igazítás miatt a memóriában a struktúra mezői közé vagy a struktúra végére igazítási lyukak, paddingek kerülhetnek
 - fordító üres bájtokat szúrhat be
- **Memóriaigénye**
 - a memória rendezés miatt a struktúra mérete lehet, hogy nagyobb mint a mezők méretének összege
 - pl: `struct Proba { char c; int i; };`
 tudjuk, hogy `char` 1 bájtos, az `int` pedig 4 bájtos ezért a memóriába így fog kinézni: `[c] [pad] [pad] [pad] [i] [i] [i] [i]`
 - azért van 3 bájtnyi padding, mert az `int` 4 bájtos és a következő szabad index ami 4-el osztható az a 4. és a 0. index a `c`-jé lesz ezért a 4. indexig paddingelni kell
 - 8 bájtot foglal a `Proba` struktúra, ebből 3 bájt padding
 - struktúra mérete optimalizálható, ha a mezőket csökkenő sorrendbe rendezzük, ezzel a beszúrt kitöltő bájtok (padding) száma minimalizálható
 - pl: először `double`, majd `int` és végül a `char` típusú mezők
- egy struktúrának saját maga nem lehet közvetlen a mezője (mert végtelen méretű lenne)
 - de önmagára mutató pointer lehet a mezője (láncolt listák alapja):
`struct ListaElem { int adat; struct ListaElem *kovetkezo; };`

- **Unió - union**

- hasonlít a struktúrához szintaktikailag, de működése alapvetően eltér, a struktúrában minden mezőnek saját memória területe van, addig **az unióban minden mező ugyanazon a memóriacímen kezdődik**
 - ezért **egyszerre csak egy mezője tartalmazhat érvényes adatot** (azt, amelyiket utoljára írtuk)
 - `union Adat { int i; float f; };`
ha beállítjuk az *i* értékét, majd kiolvassuk az *f*-et, akkor az egész szám bitjeit fogja a gép lebegőpontos számként "félreértelmezni" (type punning)
- **Memóriaigénye:** akkora, mint a legnagyobb mezője
- **Mikor alkalmazzuk?**
 - ha memóriát akarunk spórolni és tudjuk, hogy egy adat "A" típusú vagy "B" típusú, de sosem mindkettő egyszerre
 - alacsony szintű bit-elérés, azaz ugyanazt a bitsorozatot más-más típusként értelmezzük

- **Típusképzés - typedef**

- typedef kulcsszóval nem hozunk új típust létre, csak egy olvashatóbb szinonímát adunk egy már meglévőnek
 - növeli a kód hordozhatóságát és érthetőségét
- pl: `typedef unsigned long long uint64;`
- struktúráknál is kényelmes, mert a struct kulcsszó kihagyható mikor változót definiálunk

```
// Without typedef:
struct Car {
    char brand[30];
    int year;
};
```

```
// With typedef:
typedef struct {
    char brand[30];
    int year;
} Car;
```

```
typedef struct {
    int x, y;
} Pont;
```

Pont p1; // struct szó nélkül használható

Típuskonverziók

Akkor beszélünk róla, ha egy kifejezésben különböző típusú adatok szerepelnek.

1. **Implicit (automatikus) konverzió:** A fordító végzi el.
 - a. **Egész-kiterjesztés:** A char és short típusok művelet előtt int-té alakulnak
 - b. **Szokásos aritmetikai konverzió:** A cél a "legnagyobb" típus megtartása a pontosság érdekében. Sorrend: `int` -> `unsigned` -> `long` -> `float` -> `double` -> `long double`
 - c. Példa: `5 + 3.14` esetén az 5-öt a gép 5.0-ra alakítja, és az eredmény `double` lesz
2. **Explicit konverzió (Típuskényszerítés / Cast):** A programozó utasítja a gépet.
 - a. Szintaxisa: (típusnév)kifejezés
 - b. Gyakori feladat: Egész osztásból valós eredmény nyérése.
 - c. Példa: `int i=7, j=2;` esetén `(double)i / j` eredménye 3.5, míg `i / j` csak 3 lenne